

ANN Cheat Sheet

Perceptron & Neural Networks Basics

Perceptron: Single neuron that maps inputs to output via weights, bias, and activation

- **Forward propagation:** Input $\rightarrow$ weighted sum $\rightarrow$ activation $\rightarrow$ output
- **Backward propagation:** Calculate gradients with chain rule and update weights

Gradient Descent Problem: Vanishing/exploding gradients

- **Solutions:**
 - **ReLU:** $f(x) = \max(0, x)$ - Simple, fast, but "dying ReLU" problem
 - **Leaky ReLU:** $f(x) = \max(\alpha x, x)$ where α is small - Prevents dying neurons
 - **ELU:** Exponential for negative values - Smoother gradient near zero

Activation Functions

Binary Classification: Sigmoid - outputs between 0-1

Multi-class Classification: Softmax - converts logits to probabilities

Regression: Linear (no bounds), ReLU (≥ 0), TanH (-1 to 1)

Loss Functions

Regression:

- **MSE:** Standard, punishes larger errors more severely
- **MAE:** Less sensitive to outliers, constant gradient
- **Huber:** Combines MSE and MAE benefits (parameter controlled)

Classification:

- **Binary Cross-Entropy:** For two-class problems
- **Categorical CE:** For multi-class with one-hot vectors
- **Sparse Categorical CE:** For multi-class with integer labels

Optimizers

Gradient Descent: Uses full dataset, stable but slow

Stochastic GD: Single sample updates, faster but noisy

Mini-Batch GD: Balance between full and stochastic GD

SGD with Momentum: Adds velocity to reduce oscillations

Adagrad: Adapts learning rates per parameter (slows down over time)

Adadelat: Fixes Adagrad's diminishing learning rates

Adam: Combines momentum and adaptive rates (generally best)

ANN Implementation

Sequential API

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.optimizers import Adam

model = Sequential([
    Dense(128, input_shape=(input_dim,), activation='relu'),
    Dropout(0.2),
    Dense(64, activation='relu'),
    Dense(output_dim, activation='sigmoid') # sigmoid/softmax/linear based
on task
])

model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='binary_crossentropy', # or categorical_crossentropy/mse
    metrics=['accuracy']
)
```

```
history = model.fit(X_train, y_train, epochs=20, batch_size=32, validation_s  
plit=0.2)
```

Functional API

```
from tensorflow.keras.models import Model  
from tensorflow.keras.layers import Input, Dense, Dropout, concatenate  
  
# Multiple inputs example  
input_A = Input(shape=(input_dim_A,))  
input_B = Input(shape=(input_dim_B,))  
  
# Branch A  
x_A = Dense(64, activation='relu')(input_A)  
x_A = Dropout(0.2)(x_A)  
  
# Branch B  
x_B = Dense(32, activation='relu')(input_B)  
x_B = Dropout(0.2)(x_B)  
  
# Merge branches  
merged = concatenate([x_A, x_B])  
output = Dense(output_dim, activation='softmax')(merged) # sigmoid/soft  
max/linear  
  
model = Model(inputs=[input_A, input_B], outputs=output)  
  
model.compile(  
    optimizer='adam',  
    loss='categorical_crossentropy',  
    metrics=['accuracy']  
)  
  
history = model.fit(  
    [X_A_train, X_B_train],  
    y_train,  
    epochs=20,  
    batch_size=32,
```

```
validation_split=0.2  
)
```

Model Visualization and Evaluation

```
# Plot training history  
import matplotlib.pyplot as plt  
  
# Accuracy plot  
plt.figure(figsize=(12, 4))  
plt.subplot(1, 2, 1)  
plt.plot(history.history['accuracy'])  
plt.plot(history.history['val_accuracy'])  
plt.title('Model Accuracy')  
plt.ylabel('Accuracy')  
plt.xlabel('Epoch')  
plt.legend(['Train', 'Validation'], loc='lower right')  
  
# Loss plot  
plt.subplot(1, 2, 2)  
plt.plot(history.history['loss'])  
plt.plot(history.history['val_loss'])  
plt.title('Model Loss')  
plt.ylabel('Loss')  
plt.xlabel('Epoch')  
plt.legend(['Train', 'Validation'], loc='upper right')  
plt.tight_layout()  
plt.show()  
  
# Confusion matrix for classification  
from sklearn.metrics import confusion_matrix, classification_report  
import numpy as np  
import seaborn as sns  
  
y_pred = model.predict(X_test)  
y_pred_classes = np.argmax(y_pred, axis=1) # For multi-class  
# OR y_pred_classes = (y_pred > 0.5).astype(int) # For binary
```

```

cm = confusion_matrix(y_test, y_pred_classes)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()

# Detailed classification metrics
print(classification_report(y_test, y_pred_classes))

```

Model Saving and Loading

```

# Save model
model.save('my_model.h5') # Full model
model.save_weights('model_weights.h5') # Just weights

# Load model
from tensorflow.keras.models import load_model
loaded_model = load_model('my_model.h5')
# OR
model.load_weights('model_weights.h5')

```

Early Stopping

```

from tensorflow.keras.callbacks import EarlyStopping

early_stopping = EarlyStopping(
    monitor='val_loss', # or 'val_accuracy'
    patience=5, # number of epochs with no improvement
    restore_best_weights=True,
    verbose=1
)

history = model.fit(
    X_train, y_train,

```

```
epochs=100, # Maximum epochs
batch_size=32,
validation_split=0.2,
callbacks=[early_stopping]
)
```